Brightloop Subscription Management System: Project Plan

Team of 4 · Deadline 4:30 · Demo-path build

1. What we are building

One web platform that runs the full life of a subscription business for Brightloop Software: set up products and plans, turn quotes into subscriptions, generate invoices automatically, record payments, and report revenue. It replaces spreadsheets, hand-typed invoices and manual payment matching.

Goal for 4:30: a working demo of the full path, signup to dashboard, not every optional feature.

2. What the challenge needs (7 modules)

#	Module	What it must do
1	Auth and roles	Signup, login, forgot password (reset link by email), 3 roles
2	Products and variants	Name, type, sales price, cost price, notes. Variants add extra price (e.g. Odoo brand = +560)
3	Plans	Price, billing period (daily, weekly, monthly, yearly), minimum quantity, available dates, options: auto-close, closable, pausable, renewable
4	Quotation templates and subscriptions	Template, then subscription with its own number, customer, plan, start and expiry date, payment terms, lines
5	Invoices	Auto-created from subscriptions, with taxes and discounts. Never billed twice or skipped
6	Discounts and taxes	Admin-only rules. Discounts apply automatically on the next invoice

#	Module	What it must do
7	Payments and dashboard	Record payments, show paid, outstanding and overdue, revenue, active subscriptions. Every page under 2 seconds

Password rule: more than 8 characters, with an uppercase letter, a lowercase letter and a special character. Email must be unique.

3. Decisions we made

Stack

- Backend: FastAPI, SQLAlchemy, Alembic
- Database: PostgreSQL (SQLite is acceptable if it is faster to start)
- Frontend: Next.js
- Auth: JWT

Cut because of the 4:30 deadline

Cut	Instead
Celery, Redis, schedulers	A "Run billing" button/endpoint that calls the same billing function
Real email for password reset	Reset endpoint returns or prints the link (email is mocked)
PDF generation	Print-friendly invoice page using the browser print
Caching, materialized views	DB indexes, pagination, seed data
Pause and renew options	Flags are stored, only auto-close is built
Heavy documentation	One README plus this plan

Naming and API rules

- Database: `snake_case`, plural table names (`subscriptions`, `invoice_lines`), foreign keys like `customer_id`

- JSON fields: `snake_case` everywhere
- Python: `snake_case` functions, `PascalCase` classes. React: `PascalCase` components, `camelCase` variables
- API prefix `/api/v1/`, plural lowercase URLs with hyphens, actions as sub-routes (`POST /subscriptions/{id}/confirm`)
- Success: `{ "data": ... }`. List: `{ "data": [...], "page", "page_size", "total" }`. Error: `{ "error": { "code", "message" } }`
- Money: `NUMERIC(12,2)` in DB, `Decimal` in Python, 2-decimal string in JSON (`"1180.00"`). Never floats
- Dates: ISO 8601, UTC
- Overdue is **not stored**. It is computed: status is CONFIRMED, due date is before today, and balance is above 0

Shared words (use exactly these)

```
ROLES:                ADMIN, STAFF, CUSTOMER
BILLING_PERIOD:       DAILY, WEEKLY, MONTHLY, YEARLY
SUBSCRIPTION_STATUS:  DRAFT, QUOTATION, CONFIRMED, ACTIVE, CLOSED
INVOICE_STATUS:       DRAFT, CONFIRMED, PAID, CANCELLED
DISCOUNT_TYPE:        FIXED, PERCENTAGE
PAYMENT_METHOD:       CASH, CARD, BANK_TRANSFER, UPI, OTHER
```

Code rules

- Layers: routers (HTTP only), then services (business logic), then models (DB)
- All money math lives in one file, `services/billing.py`
- Role checks use one shared guard, e.g. `require_role("ADMIN")`
- Customer endpoints take `customer_id` from the login token, never from the request
- Every list endpoint is paginated
- No secrets in git, keep `.env.example` current
- `main` always runs. Small commits, merge often, nobody pushes a broken `main` in the last hour

4. User types and access

There are exactly **3 user types**.

User type	Who they are	How they get an account
ADMIN	Controls the whole system	Created at setup (seeded)
STAFF	Internal team doing day-to-day work, including the finance lead	Only an admin can add staff
CUSTOMER	Business buying subscriptions	Signs up themselves

Access matrix

Action	ADMIN	STAFF	CUSTOMER
Add or remove staff	Yes	No	No
Create products, variants, plans	Yes	No	No
Create taxes and discount rules	Yes	No	No
Create quotation templates	Yes	No	No
View products and plans	Yes	Yes	Yes (read only)
Create customer records	Yes	Yes	No
Create subscription from a template	Yes	Yes	No
Move subscription through statuses	Yes	Yes	No
View all subscriptions and invoices	Yes	Yes	Own only
Confirm, cancel, send, print invoices	Yes	Yes	Print own only
Run billing	Yes	Yes	No
Record payments	Yes	Yes	No
Finance dashboard	Yes	Yes	No
View own subscriptions and invoices	n/a	n/a	Yes
Close own subscription	n/a	n/a	Yes (assumption, see section 9)

5. Screens (windows) and page count

Expected total: **21 pages**. If forms are built as pop-ups instead of separate pages, this drops to about 15.

Public pages (4), everyone

1. Login
2. Signup
3. Forgot password
4. Reset password

Admin-only pages (6)

5. Staff management (list and add staff)
6. Products (list, create, edit, with variants and extra prices)
7. Plans (list, create, edit, with billing period, min quantity, dates, options)
8. Taxes (list, create, edit)
9. Discounts (list, create, edit)
10. Quotation templates (list, create, edit)

Admin and staff pages (7)

11. Finance dashboard (active subscriptions, revenue, outstanding, overdue list, filters)
12. Customers (list, create)
13. Subscriptions list (filter by status)
14. Subscription form and detail (lines, status buttons)
15. Invoices list (filter by status, overdue)
16. Invoice detail (confirm, cancel, send, print, record payment)
17. Payments list

Customer portal pages (4)

18. My subscriptions
19. Subscription detail
20. My invoices
21. Invoice detail and print

Pages per user type

User type	Pages they can open
Not logged in	4
CUSTOMER	4 public + 4 portal = 8
STAFF	4 public + 7 shared = 11
ADMIN	4 public + 6 admin-only + 7 shared = 17

6. Flow of the software

Big picture

```
ADMIN SETUP
  Products + Variants -> Plans -> Taxes -> Discounts -> Quotation Templates
          |
          v
STAFF / SALES
  Pick template -> create Subscription for a customer
  DRAFT -> QUOTATION -> CONFIRMED -> ACTIVE
          |
          v
BILLING DAY ("Run billing" button)
  Find ACTIVE subscriptions that are due
  -> create invoice (lines + auto discount + tax)
  -> DRAFT -> CONFIRMED (send / print)
  -> move next billing date forward
          |
          v
PAYMENTS
  Record payment (method, amount, date, note) on the invoice
  -> fully paid = PAID, otherwise still outstanding
  -> past due date and unpaid = overdue (computed)
          |
          v
FINANCE DASHBOARD
  Active subscriptions, revenue, outstanding, overdue, filters
```

Customer journey

Signup, then login, then the portal shows only their own subscriptions and invoices. If they forget the password, the reset link brings them back in.

Subscription status flow

DRAFT, then QUOTATION, then CONFIRMED, then ACTIVE, then CLOSED. Everyone on the team can see the current status on the subscription list.

Invoice status flow

DRAFT, then CONFIRMED, then PAID. An invoice can be CANCELLED before it is paid.

7. Billing rules

1. **Which subscriptions:** status is ACTIVE and next billing date is today or earlier.
2. **Line amount:** quantity multiplied by unit price. Unit price is the plan price plus any variant extra (e.g. +560 for the Odoo brand).
3. **Discount first:** apply any valid discount (fixed or percentage). It must be within its start and end dates, meet the minimum purchase or quantity, and be under its usage limit. It stops automatically when expired or used up.
4. **Then tax:** (line amount minus discount) multiplied by the tax rate set by the admin.
5. **Total:** line amount minus discount plus tax. Round half up to 2 decimals per line.
6. **No double billing:** a unique rule on subscription and billing period start means running billing twice creates nothing new.
7. **No skipping:** after invoicing, the next billing date moves forward by the plan period (daily, weekly, monthly or yearly).
8. **Payments:** when payments on an invoice reach the total, it becomes PAID. Otherwise it stays outstanding.

8. Team split

Person	Build, in this order
P1	Models and enums, auth and role guard, billing function (tax, discount, no duplicates)

Person	Build, in this order
P2	Products, variants, plans, taxes, discounts, templates, then subscriptions and statuses, then payments
P3	Login page, admin and staff screens: catalog, subscription form, invoice list, payment form
P4	Customer portal, dashboard, seed script, README

Hand-offs: P1 pushes the models first because everyone depends on them. P2 gives P1 a sample subscription so billing can be tested early. P3 and P4 use mock data until the API exists.

9. Assumptions to confirm

- Customers can view and close their own subscription. The challenge says customers "manage" it but does not say exactly how.
- The finance lead is a STAFF user, so the dashboard is open to STAFF and ADMIN.
- Staff can create customer records for large customers, while normal customers sign up themselves.
- Invoice CANCELLED is included although the challenge lists only Draft, Confirmed, Paid, because the team must be able to cancel an invoice.

10. Timeline to 4:30

When	Goal
First 30 minutes	Agree on enums and endpoint list. P1 pushes models.
Until 90 minutes before deadline	Each person builds their own part
90 to 45 minutes before	Merge everything, run the full demo flow, fix what breaks
45 to 20 minutes before	**Feature freeze.** Bug fixes, seed data, README

When	Goal
Last 20 minutes	Run the demo once from fresh `main`, write down demo logins, submit

If a feature is not working 45 minutes before the deadline, cut it.

11. Demo checklist

- [ ] Signup with weak password is rejected, duplicate email is rejected
- [ ] Admin creates product with variants, plan, tax and discount
- [ ] Staff creates subscription from a template and moves it DRAFT to ACTIVE
- [ ] "Run billing" creates invoices with tax and discount applied
- [ ] Running billing a second time creates no duplicates
- [ ] Payment recorded, invoice becomes PAID
- [ ] Customer portal shows only that customer's data
- [ ] Dashboard shows active subscriptions, revenue, outstanding, overdue
- [ ] Pages respond in under 2 seconds with seed data
- [ ] Demo logins for ADMIN, STAFF and CUSTOMER written in the README